

Alemeno Marker Scanner - Approach

Marker choice

The app uses the provided Marker 1. It is a square black border with a single filled black finder block near the top-left corner. The remaining interior is mostly empty, which leaves room for encoded content and makes the marker easy to distinguish from ordinary square objects.

Camera flow

The React Native app renders a live back-camera preview with a 1:1 capture ratio. On supported Android devices it selects a square picture size between 2000x2000 and 3000x3000 pixels, preferring the size closest to 2500x2500. The scan loop captures up to 60 frames and stops once 20 valid marker crops are produced.

Detection

Each JPEG frame is decoded in JavaScript, thresholded for true black pixels, and scanned for the largest square-like connected component. The detector extracts the four extreme corners of that component, validates that all four outer edges are continuous, and perspective-warps the candidate into a 300x300 image.

Orientation correction

After warping, the detector measures four corner finder zones. The crop is rotated in 90-degree steps until the filled finder block is in the top-left location. The final image is encoded as JPEG and displayed exactly at 300x300 pixels.

False-positive rejection

Validation checks the four black border bands, the expected top-left finder block, the absence of finder blocks in the other corners, low central black density, and the red error mark used in the supplied incorrect samples. The included test runner verifies that the three correct Marker 1 images are accepted and the four incorrect Marker 1 images are rejected.

Performance

Candidate detection runs on a downsampled frame for speed. Only the accepted square candidate is warped at 300x300 resolution, keeping per-frame work small enough for the assignment target of a sub-3000 ms scan-to-result path on a typical Android device.